

Programmation orientée objet (UAA14)

Exercices : série 6

Objets d'apprentissage :

- ✓ Modéliser une logique de programmation orientée objet.
- ✓ Déclarer une classe.
- ✓ Instancier une classe.
- ✓ Utiliser les méthodes de l'objet instancié.
- ✓ Traduire un algorithme dans un langage de programmation.
- ✓ Commenter des lignes de code.
- ✓ Tester le programme conçu.
- ✓ Caractériser les attributs dans une classe (encapsulation).
- ✓ Caractériser les méthodes dans une classe (encapsulation).
- ✓ Décrire la création d'un constructeur.
- ✓ Extraire d'un cahier des charges les informations nécessaires à la programmation.
- ✓ Programmer en recourant aux instructions et types de données nécessaires au développement d'une application.
- ✓ Corriger un programme défaillant.
- ✓ Développer une classe sur la base d'un cahier des charges en respectant le paradigme de la programmation orientée objet.
- ✓ Programmer en recourant aux classes nécessaires au développement d'une application orientée objet.
- ✓ Améliorer un programme pour répondre à un besoin défini.

Exercice 1 : personne

Étape 1

Crée une classe `Personne` caractérisée par un nom, un prénom, un sexe et une date de naissance. Les dates ne sont pas au programme : range la date de naissance sous la forme d'une chaîne de caractères, par exemple `"2008-03-14"`.

Étape 2

Crée un setter / getter pour chaque variable d'instance.

Étape 3

Prévois un constructeur (à 6 arguments) qui permet de créer des personnes à partir d'un nom, un prénom, un sexe, une année, un mois et un jour de naissance. C'est le constructeur qui assemble ces trois nombres en une seule chaîne.

Étape 4

Mets à jour le setter du sexe de telle sorte qu'il génère une exception de type `SexeIncorrectException` lorsque le sexe renseigné n'est pas correct. Les seules valeurs permises pour le sexe sont `'m'`, `'M'`, `'f'`, `'F'`, `'x'` et `'X'`.

N'oublie pas de définir toi-même la classe `SexeIncorrectException` : une seule ligne suffit. Profites-en pour lui donner un message explicite au moment du `throw` – c'est ce message que tu afficheras à l'étape 7.

Étape 5

Fais en sorte que le constructeur propage l'erreur du setter. En effet, ce n'est pas à lui de gérer cette exception, mais au script PHP qui utilise le constructeur.

Étape 6

Prévois également la méthode `__toString()` (celle vue au cours, avec ses deux tirets bas) qui retourne le prénom suivi du nom.

Étape 7

Réalise les deux étapes suivantes :

1. Essaie (**try**) de créer une personne avec un sexe incorrect
2. Capture (**catch**) l'exception et affiche à l'écran le message correspondant à l'erreur

Étape 8

Mets à jour ton code :

- lors de la création (réussie) d'une personne, un message de confirmation doit apparaître à l'écran ;
- lorsque la création lance une exception (échoue), un message d'erreur doit apparaître à l'écran.

Exercice 2 : streamers

L'objectif de cet exercice est de simuler une connexion à une base de données contenant des informations sur des Streamers.

Étape 1

Crée une classe `Streamer` permettant de créer des streamers. Un streamer est caractérisé par :

- son pseudo ;
- sa plateforme de jeu (PC, PS5, XBOX, etc.) ;
- son nombre d'abonnés.

Prévois un constructeur pour cette classe ainsi que des getters/setters.

Étape 2

Crée une classe `ConnexionBD` modélisant une connexion à une base de données. Les informations nécessaires pour se connecter à une base de données sont :

- une adresse (host) ;
- un nom d'utilisateur (identifiant) ;
- un mot de passe ;
- un tableau contenant un ensemble de streamers (autrement dit, les données).

Prévois également un constructeur : lorsqu'une `ConnexionBD` est instanciée, quelques streamers sont automatiquement ajoutés au tableau de streamers (pour simuler une vraie base de données).

Étape 3

Il est possible que l'établissement d'une connexion à une base de données échoue, et ce pour plusieurs raisons :

- l'adresse renseignée ne redirige vers aucune base de données ;
- le nom d'utilisateur n'existe pas ;
- le mot de passe est incorrect.

Fais en sorte que le constructeur de la classe `ConnexionBD` lance une exception `ConnexionException` dans les cas suivants :

- l'adresse renseignée n'est pas `localhost:3306/streamers`
- le nom d'utilisateur n'est pas `admin`
- le mot de passe n'est pas `root`

Étape 4

Ajoute les méthodes suivantes à la classe `ConnexionBD` :

- `getStreamer` : retourne le streamer identifié par le pseudo reçu en argument
- `getAllStreamers` : retourne un tableau contenant l'ensemble des streamers présent dans la base de données

- `addStreamer` : ajoute le streamer reçu en argument dans le tableau des streamers
- `removeStreamer` : supprime le streamer du tableau identifié par le pseudo reçu en argument

Attention, des exceptions doivent être lancées dans certains cas :

- toutes les méthodes lancent possiblement une `ConnexionException`, dans le cas où la connexion avec la base de données est interrompue (ex. coupure internet) ;
- les méthodes `getStreamer`, `getAllStreamers` et `removeStreamer` lancent possiblement une `NoResultException` dans le cas où aucun streamer n'est trouvé dans la base de données ;
- la méthode `addStreamer` lance possiblement une `StreamerAlreadyExistsException` dans le cas où on tente d'ajouter un streamer qui existe déjà (deux pseudos identiques).

Une coupure réseau ne se produira évidemment pas pendant tes essais : il faut donc pouvoir la **simuler**. Ajoute pour cela à la classe `ConnexionBD` une variable d'instance booléenne `$connectee`, mise à vrai dès que le constructeur a réussi, ainsi qu'une méthode `deconnecter` qui la passe à faux. Chaque méthode vérifie cette variable avant de travailler et lance une `ConnexionException` si la connexion est perdue.

Tu pourras alors écrire un test du genre :

```
$bd = new ConnexionBD("localhost:3306/streamers", "admin", "root");
$bd->deconnecter();
$bd->getAllStreamers(); // doit lancer une ConnexionException
```

Teste l'ensemble de tes méthodes : pour chacune, un appel qui réussit, puis chacun de ses cas d'erreur. Ton script de test doit au moins provoquer :

- une `ConnexionException` lors de la création, avec un mot de passe incorrect ;
- une `NoResultException` en cherchant un pseudo qui n'existe pas avec `getStreamer` ;
- une `StreamerAlreadyExistsException` en ajoutant deux fois le même streamer ;
- une `NoResultException` en supprimant un pseudo qui n'existe pas ;
- une `NoResultException` en appelant `getAllStreamers` après avoir supprimé tous les streamers ;
- une `ConnexionException` en appelant une méthode après `deconnecter`.

Pour chaque cas, affiche le message de l'exception attrapée.

Exercice 3 : gestionnaire de fichiers

L'objectif de cet exercice est de créer un gestionnaire de fichier qui permet d'écrire et de lire dans un fichier.

Étape 1

Crée la classe `GestionnaireFichier` permettant d'interagir avec des fichiers.

Étape 2

Ajoute à la classe `GestionnaireFichier` la méthode `lireFichier` qui retourne le contenu d'un fichier localisé au chemin reçu en argument.

Pour lire un fichier ligne par ligne, utilise la classe `LecteurFichier` disponible à l'adresse :

<https://github.com/LucasEmbrechts/UAA14-POO-PHP/tree/main/OperationsFichier>

Attention : des cas d'erreur (exceptions) sont à prévoir :

- Si le fichier n'existe pas, le constructeur de la classe `LecteurFichier` lance une `LectureException`
- En cas d'erreur de lecture (ex. le fichier est supprimé pendant que le programme lit les lignes), la fonction `lireLigne` lance également une `LectureException`

Pour savoir s'il reste des lignes à lire, la classe `LecteurFichier` propose aussi la méthode `encoreUneLigneALire`, qui retourne un booléen.

Fais en sorte que la méthode `lireFichier` lance une exception nommée `FichierInexistantException` contenant le message « Le fichier <chemin> n'existe pas ou a été supprimé » lorsqu'une de ces deux erreurs est détectée.

Étape 3

Ajoute à la classe `GestionnaireFichier` la méthode `ecrireDansFichier` qui écrit une chaîne de caractères dans un fichier situé au chemin spécifié. La chaîne de caractères et le chemin sont tous les deux reçus en argument.

Pour écrire une ligne dans un fichier, utilise la classe `EcritureFichier` disponible à l'adresse :

<https://github.com/LucasEmbrechts/UAA14-POO-PHP/tree/main/OperationsFichier>

Attention : des cas d'erreur (exceptions) sont à prévoir :

- si le fichier n'existe pas ou n'est pas accessible en écriture, le constructeur de la classe `EcritureFichier` lance une `LectureException` ;
- si l'écriture elle-même échoue, la méthode `ecrire` lance une `EcritureException`. Le fichier `EcritureFichier.class.php` n'inclut pas cette classe d'exception : pense à inclure toi-même `EcritureException.class.php`.

Fais en sorte que la méthode `ecrireDansFichier` lance une exception nommée `AccesFichierException` contenant le message « Accès refusé au fichier <chemin> » lorsqu'une de ces deux erreurs est détectée.

Étape 4

Teste les deux méthodes de la classe `GestionnaireFichier` en essayant notamment de :

- lire le contenu d'un fichier inexistant ;
- écrire dans un fichier verrouillé (pour en créer un, crée un fichier puis rends-le accessible en lecture seule : clic droit, puis Propriétés ou Lire les informations).

Exercice 4 : trois couches, trois rôles

Dans une application web, les erreurs ne naissent pas toutes au même endroit. Certaines viennent des données (un membre introuvable), d'autres des règles de l'application (un pseudo déjà pris), d'autres encore de la technique (la base de données ne répond plus).

Une application bien construite range son code en trois couches, et chacune a son rôle face aux exceptions :

- la couche **d'accès aux données** lit et écrit les données ; elle lance une exception quand une donnée manque ou quand les données sont inaccessibles ;
- la couche **logique** applique les règles de l'application ; elle lance une exception quand une règle est violée ;
- le **script d'affichage** essaie, attrape et affiche. C'est le seul endroit du programme où l'on trouve des `echo`.

Étape 1

Reprends la classe `Membre` de l'exercice 2 de la série 3 (Pseudo, Email, Points, Admin, Id, avec ses arguments facultatifs et ses méthodes `estAdmin` et `ajouterPoints`). Elle ne change pas.

Étape 2

Crée la classe `AccesMembres`. Son constructeur reçoit un tableau de lignes associatives, qu'il range dans une variable d'instance privée :

```
$lignes = array(
    array("Id" => 1, "Pseudo" => "zoe",    "Email" => "zoe@mail.be",
    "Points" => 42, "Admin" => false),
    array("Id" => 2, "Pseudo" => "malik", "Email" => "malik@mail.be",
    "Points" => 17, "Admin" => false),
    array("Id" => 3, "Pseudo" => "sam",    "Email" => "sam@mail.be",
    "Points" => 63, "Admin" => true)
);
$acces = new AccesMembres($lignes);
```

Ajoute la méthode `obtenirMembre` qui reçoit un pseudo et retourne l'objet `Membre` construit à partir de la ligne correspondante. Si aucune ligne ne correspond, elle lance une `MembreIntrouvableException` contenant le message « Aucun membre nommé <pseudo> ».

Étape 3

Ajoute la méthode `enregistrerMembre` qui reçoit un objet `Membre` et l'ajoute aux lignes sous la forme d'un tableau associatif. C'est à ce moment que le membre reçoit son identifiant : le nombre de lignes + 1.

Étape 4

Crée la classe `Logique`. Son constructeur reçoit un objet `AccesMembres` et le range dans une variable d'instance privée.

Ajoute-lui la méthode `inscrire`, qui reçoit un pseudo et une adresse mail. Un pseudo ne peut être pris qu'une seule fois : si un membre porte déjà ce pseudo, la méthode lance une

`PseudoDejaPrisException` contenant le message « Le pseudo <pseudo> est déjà pris » ; sinon, elle crée le membre et le fait enregistrer.

Attention : la classe `AccesMembres` ne possède aucune méthode qui teste l'existence d'un membre, et tu ne dois pas lui en ajouter. Tu dois donc te servir de `obtenirMembre...` et de son exception.

Indice : essaie d'obtenir le membre. Si tu y arrives, que peux-tu en conclure ? Et si tu n'y arrives pas ?

Réponds ensuite à la question suivante : dans ta méthode `inscrire`, la `PseudoDejaPrisException` est-elle lancée à l'intérieur d'un bloc `try` ? Si oui, pourquoi n'est-elle pas attrapée par le `catch` qui le suit ?

Étape 5

Ajoute à la classe `Logique` la méthode `ouvrirAdministration`, qui reçoit un pseudo et retourne le membre correspondant s'il est administrateur. Sinon, elle lance une `AccesRefuseException` contenant le message « <pseudo> n'est pas administrateur ».

Si le membre n'existe pas, cette méthode n'a rien de particulier à faire : la `MembreIntrouvableException` doit arriver telle quelle jusqu'au script d'affichage.

Étape 6

Écris le script d'affichage. Il crée les objets nécessaires, puis essaie, dans l'ordre :

- d'inscrire lea (lea@mail.be) ;
- d'inscrire zoe (zoe2@mail.be) ;
- d'ouvrir l'administration pour sam, puis pour malik, puis pour tom.

Chaque action a son propre `try`, suivi d'autant de `catch` que nécessaire. Le script ne contient **aucun** `if` : toutes les décisions sont prises dans les classes. Le résultat attendu est le suivant :

```
Inscription de lea : réussie
Inscription de zoe : refusée – Le pseudo zoe est déjà pris
Administration pour sam : accès accordé
Administration pour malik : refusée – malik n'est pas administrateur
Administration pour tom : refusée – Aucun membre nommé tom
```

Vérifie enfin que lea a bien reçu l'identifiant 4.

Étape 7

Reste l'erreur technique. Reprends le mécanisme de l'exercice 2 : ajoute à `AccesMembres` une variable d'instance `$connectee` et une méthode `deconnecter`. Quand la connexion est perdue, `obtenirMembre` et `enregistrerMembre` lancent une `DonneesIndisponiblesException` contenant le message « les données sont indisponibles ».

Dans le script d'affichage, appelle `$acces->deconnecter()` juste avant la dernière action. La dernière ligne devient :

```
Administration pour tom : erreur technique – les données sont indisponibles
```

Une erreur technique ne se traite pas comme une erreur de l'utilisateur : place son `catch` dans un second `try` qui entoure le premier (voir la diapo « Un `try` dans un `try` »).

Réponds ensuite aux deux questions suivantes :

- as-tu dû modifier la classe `Logique` ? pourquoi ?
- dans `inscrire`, pourquoi est-il important d'attraper une `MembreIntrouvableException` et pas simplement une `Exception` ?

Étape 8

Range ton programme : une classe par fichier (`Membre.class.php`, `AccesMembres.class.php`, `Logique.class.php`), les classes d'exception dans un dossier `exceptions`, et le script d'affichage dans un fichier à part. Chaque fichier inclut (`include_once`) uniquement ce dont il a besoin.

À retenir : c'est l'organisation d'une application web réelle — un dossier pour l'accès aux données, un pour la logique, un pour l'affichage. Les exceptions sont le moyen par lequel les couches du bas préviennent les couches du haut, sans jamais rien afficher elles-mêmes.

Exercice 5 : les pronostics

Règle du jeu

À partir d'ici, on ne te donne plus d'étapes. Ni la liste des classes, ni celle des méthodes, ni celle des exceptions : c'est à toi de les décider. Tu ne reçois que la situation à modéliser, quelques exigences, et le résultat attendu à l'écran.

La situation

Une application permet aux membres de pronostiquer le score des rencontres d'un tournoi. Voici, telles quelles, les lignes que la base de données renvoie pour les tables `rencontre` et `pari` :

```
$rencontres = array(
    array("Id" => 1, "Locaux" => "Belgique", "Visiteurs" => "Maroc",
"Commencee" => true),
    array("Id" => 2, "Locaux" => "Japon",      "Visiteurs" => "Brésil",
"Commencee" => false),
    array("Id" => 3, "Locaux" => "Canada",    "Visiteurs" => "Croatie",
"Commencee" => false)
);
$paris = array(
    array("Pseudo" => "zoe", "IdRencontre" => 2, "ButsLocaux" => 1,
"ButsVisiteurs" => 3)
);
```

Un pronostic est refusé si la rencontre n'existe pas, si elle a déjà commencé, si le membre a déjà pronostiqué cette rencontre, ou si l'un des deux scores est négatif.

Voici les pronostics que les membres tentent d'enregistrer, dans cet ordre (pseudo, rencontre, buts des locaux, buts des visiteurs) :

```
$essais = array(
    array("zoe",    3,  2, 0),
    array("zoe",    2,  1, 1),
    array("malik",  1,  0, 2),
    array("malik",  9,  1, 0),
    array("sam",    3, -1, 2),
    array("sam",    3,  2, 2)
);
```

Les exigences

- trois couches, comme dans l'exercice 4 : accès aux données, logique, affichage ;
- une classe par table, construite à partir des lignes ;
- chaque motif de refus a sa propre classe d'exception, et chaque exception transporte son message ;
- un pari au score négatif ne peut pas exister, même un instant ;
- le script d'affichage ne contient aucun `if` : il essaie, il attrape, il affiche.

Le résultat attendu

Les espaces qui alignent les colonnes ne sont pas exigés.

```
--- PRONOSTICS ---
zoe  → rencontre 3 (2 - 0) : enregistré
zoe  → rencontre 2 (1 - 1) : refusé, zoe a déjà pronostiqué cette
rencontre
malik → rencontre 1 (0 - 2) : refusé, la rencontre Belgique - Maroc a déjà
commencé
malik → rencontre 9 (1 - 0) : refusé, aucune rencontre n° 9
sam  → rencontre 3 (-1 - 2) : refusé, un score ne peut pas être négatif
sam  → rencontre 3 (2 - 2) : enregistré

--- PRONOSTICS ENREGISTRÉS ---
zoe : Japon 1 - 3 Brésil
zoe : Canada 2 - 0 Croatie
sam : Canada 2 - 2 Croatie
```

Pour vérifier

- quelle exception est lancée par un constructeur ? que vaut la variable qui devait recevoir l'objet lorsque cela se produit ?
- tes `catch` font sans doute tous la même chose : pourrais-tu les remplacer par un seul ? Que perdrais-tu ? Dans une vraie page web, pourquoi voudrait-on réagir différemment selon le motif du refus ?
- si la base de données tombait en panne pendant les essais, quelle couche lancerait l'exception, et où l'attraperais-tu ?

Exercice 6 : la page d'inscription

Jusqu'ici, tes scripts d'affichage écrivaient directement leurs résultats. Dans une application web, l'affichage est une page : elle reçoit un formulaire, appelle la logique, et affiche dans le HTML un message qui dépend de l'exception attrapée. C'est le rôle de la troisième couche de l'exercice 4.

Étape 1

Reprends les fichiers de l'exercice 4 (étape 8) et ajoute un dossier `vue` qui contiendra la page `inscription.php`.

Étape 2

Dans `inscription.php`, crée un formulaire qui demande un pseudo et une adresse mail, et qui s'envoie en méthode POST vers la page elle-même.

Étape 3

Les membres inscrits doivent survivre d'un affichage de la page à l'autre. Au premier affichage, crée l'objet `AccesMembres` à partir des trois lignes de l'exercice 4 et range-le en session ; aux affichages suivants, reprends celui qui se trouve en session.

Attention : pour qu'un objet rangé en session soit correctement relu à la page suivante, PHP doit connaître sa classe avant l'appel à `session_start()`. Place donc les inclusions tout en haut de la page, avant le `session_start()`. Sans cela, tes objets ressortiront de la session vidés de leur contenu, sans le moindre message d'erreur.

Étape 4

Quand le formulaire est reçu, la page appelle la méthode `inscrire` dans un `try`. Les `catch` ne font aucun `echo` : chacun range un message dans une variable `$message`, que la page affiche plus bas, au-dessus du formulaire.

- inscription réussie : « Bienvenue, <pseudo> ! » ;
- pseudo déjà pris : le message de l'exception ;
- données indisponibles : « Le service est momentanément indisponible, réessaie plus tard. »

Pour pouvoir tester le dernier cas, ajoute un bouton « Simuler une panne » qui appelle `deconnecter` sur l'objet rangé en session.

Étape 5

Ajoute une règle dans la classe `Logique` : un pseudo doit contenir au moins 3 caractères. Sinon, `inscrire` lance une `PseudoInvalideException` contenant le message « Un pseudo doit contenir au moins 3 caractères ». Adapte la page pour afficher ce message.

Étape 6

Sous le formulaire, affiche la liste des membres inscrits (pseudo et points). Pour cela, ajoute à `AccesMembres` une méthode `listeMembres` qui retourne un tableau d'objets `Membre` construit à partir des lignes, comme dans l'exercice 2 de la série 3, et à `Logique` une méthode qui la relaie. Si les données sont indisponibles, la page affiche « Liste indisponible » à la place de la liste.

Étape 7

Réponds aux questions suivantes :

- pour l'erreur technique, pourquoi la page n'affiche-t-elle pas simplement le message de l'exception ?
- dans quels fichiers de ton programme trouve-t-on maintenant des `echo` ?
- la page contient des `if`. L'un d'eux sert-il à détecter une erreur ?

À retenir : une page web ne décide de rien. Elle transmet la demande à la logique, attrape ce qui revient, et choisit comment le montrer à l'utilisateur.

Références

Les présents exercices ont été élaborés à l'aide des ressources suivantes :

- <https://www.pierre-giraud.com/php-mysql-apprendre-coder-cours/introduction-programmation-orientee-objet/>
- Cours de « Développement d'applications WEB », Hénallux (2019)